HashMath: A Content-Addressed Calculus of Inductive Constructions for Permissionless Formal Mathematics

Samuel Schlesinger

Abstract

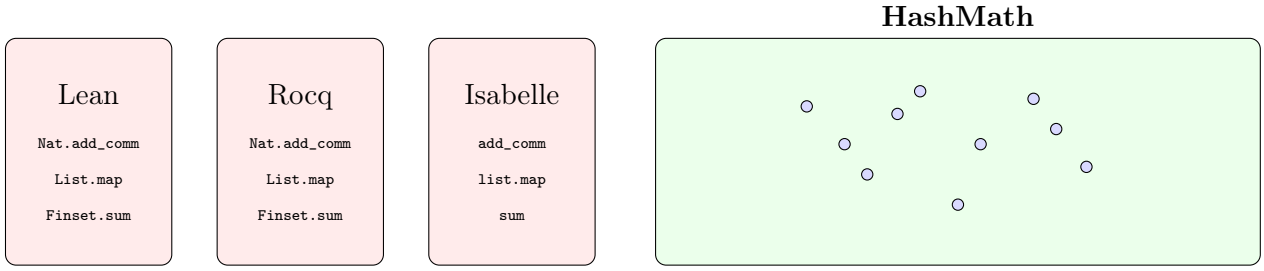
Formalized mathematics is fragmented across incompatible, name-dependent systems. Lean, Rocq, and Isabelle each maintain curated libraries where every definition and theorem is identified by a human-chosen name, creating coordination bottlenecks that limit the scale at which humans and AI can contribute. We observe that mathematical identity is structural, not nominal: two proof terms with identical structure are the same proof regardless of what anyone calls them. We propose **HashMath**, a content-addressed Calculus of Inductive Constructions (CIC) in which every well-typed term is indexed by its SHA3 hash, with all dependencies referenced by hash rather than by name. The result is a global, permissionless, append-only knowledge base where correctness is guaranteed by type theory, names are optional metadata, and contributions require no coordination, review, or naming consensus. A type-to-inhabitants index enables type-directed search across the entire knowledge base, allowing AI agents and human mathematicians to discover relevant lemmas by their type signatures alone. HashMath provides the infrastructure for Tao's vision of collaborative human-AI mathematics at scale: 1000 agents and 100 mathematicians contributing simultaneously, building on each other's work by hash, without a single naming conflict.

1 Introduction

Formalized mathematics is at an inflection point. Mathlib, the community-driven library for Lean 4, now contains over 210,000 theorems and continues to grow at an accelerating pace [2]. Google DeepMind's AlphaProof achieved silver-medal performance at the 2024 International Mathematical Olympiad [3], and DeepSeek-Prover-V2 has reached gold-level IMO performance [4]. Terence Tao envisions a near future in which "big theorems are proved by 20 people and multiple AIs, each proving little things" [5].

Yet the infrastructure for this vision does not exist. Today's proof assistants—Lean [1], Rocq (formerly Coq), Agda, Isabelle/HOL—organize knowledge through *named references* within *curated repositories*. Every definition, lemma, and theorem must be given a unique name, reviewed by maintainers, and placed within a coherent namespace hierarchy. This works when a small community curates a single library, but it fundamentally limits scale.

The coordination bottleneck. Contributing to Mathlib requires conforming to naming conventions, passing linters, and waiting for human review. These are reasonable quality controls, but they create a throughput ceiling. When AI systems can generate thousands of correct proofs per hour, the review queue becomes the bottleneck, not the proving. Independent formalizations cannot compose without reconciliation of names, import paths, and stylistic conventions.



(a) Fragmented silos: duplicated work, incompatible names

(b) Unified pool: one hash per proof

Figure 1: Formalized mathematics today exists in isolated silos with duplicated effort and incompatible naming. HashMath provides a single content-addressed pool where every unique proof term has exactly one identity.

The naming problem. Mathematicians disagree on names. Is it `Nat.add_comm` or `plus_comm` or `addition_commutative`? Different communities, different languages, and different traditions choose differently. AI agents, which operate at machine speed, generate names like `theorem_7a3f`—functional but meaningless to humans. When two independent groups formalize the same theorem under different names, reconciliation requires human labor to identify the duplication, choose a canonical name, and redirect references.

The Polymath precedent. In 2009, Timothy Gowers launched the Polymath project, demonstrating that permissionless mathematical collaboration works [6]. Over 40 contributors proved the density Hales-Jewett theorem through blog comments, with the result published in the *Annals of Mathematics* [7]. But the coordination was ad hoc—blog comments, manual tracking, human synthesis. HashMath provides the infrastructure that Polymath lacked: a system where contributions compose automatically, correctness is verified mechanically, and no central coordinator is required.

Core insight. Mathematical objects have *intrinsic identity* determined by their structure. The proof that $n + 0 = n$ by induction on n is the same proof regardless of whether it is called `Nat.add_zero`, `plus_n_O`, or `h_7f3a2b`. Names are metadata—useful for human navigation but irrelevant to mathematical content.

We propose **HashMath**: a content-addressed Calculus of Inductive Constructions in which every well-typed term is identified by its cryptographic hash. Dependencies are referenced by hash, not by name. The result is a global, append-only knowledge base that requires no coordination to use, no curator to maintain, and no naming consensus to function.

> *Bitcoin solved double-spending without trusted third parties. HashMath solves naming conflicts without trusted curators.*

2 The Calculus of Inductive Constructions

We briefly present the Calculus of Inductive Constructions (CIC), the type theory underlying both Lean 4 and Rocq. Our presentation follows Coquand and Huet [8], Coquand and Paulin-Mohring [9], and Werner [10].

2.1 Terms and Types

CIC terms are defined by the following grammar, using de Bruijn indices for bound variables:

$$t, T \quad ::= \quad x \mid c \mid \mathsf{Sort}(u) \mid t\ t \mid \lambda(x{:}T).\, t \mid \Pi(x{:}T).\, T \mid \mathsf{let}\ x{:}T := t\ \mathsf{in}\ t$$

where:

- x denotes a variable (represented internally as a de Bruijn index $i \in \mathbb{N}$),
- c denotes a constant (a reference to a previously defined declaration),
- $\mathsf{Sort}(u)$ is a universe sort at level u,
- $t\ t$ is application, $\lambda(x{:}T).\, t$ is abstraction, $\Pi(x{:}T).\, T$ is the dependent product,
- $\mathsf{let}\ x{:}T := t\ \mathsf{in}\ t$ is a local definition.

Lean 4's kernel type `Expr` is a concrete realization of this grammar, with additional fields for binder names and binder info (implicit/explicit annotations) that are relevant for elaboration but not for the core type theory [1].

2.2 Typing Rules

We present the key typing judgments. A context Γ is a list of typed variable declarations. The judgment $\Gamma \vdash t : T$ asserts that term t has type T in context Γ.

$$\frac{}{\Gamma \vdash \mathsf{Sort}(i) : \mathsf{Sort}(i+1)}\ \textsc{Sort} \qquad \frac{(x : T) \in \Gamma}{\Gamma \vdash x : T}\ \textsc{Var} \qquad \frac{\Gamma \vdash A : \mathsf{Sort}(i) \qquad \Gamma, x{:}A \vdash B : \mathsf{Sort}(j)}{\Gamma \vdash \Pi(x{:}A).\, B : \mathsf{Sort}(\max(i,j))}\ \textsc{Pi}$$

$$\frac{\Gamma \vdash A : \mathsf{Sort}(i) \qquad \Gamma, x{:}A \vdash t : B}{\Gamma \vdash \lambda(x{:}A).\, t : \Pi(x{:}A).\, B}\ \textsc{Lam} \quad \frac{\Gamma \vdash f : \Pi(x{:}A).\, B \qquad \Gamma \vdash a : A}{\Gamma \vdash f\ a : B[a/x]}\ \textsc{App} \quad \frac{\Gamma \vdash t : T \qquad T \equiv_\delta T'}{\Gamma \vdash t : T'}\ \textsc{Conv}$$

The conversion rule (Conv) allows replacing a type with a definitionally equal one. We refer to Coquand and Paulin-Mohring [9] for the complete set of rules including the let-binding rule, and to the Lean 4 kernel source for a modern implementation [1].

2.3 Reductions

Definitional equality $\equiv_\delta$ is generated by:

- **β-reduction**: $(\lambda(x{:}A).\, t)\ a \ \longrightarrow_\beta\ t[a/x]$
- **δ-reduction**: unfolding a named constant to its definition
- **ι-reduction**: reducing pattern matches on constructors
- **ζ-reduction**: substituting let-bindings, $\mathsf{let}\ x{:}T := v\ \mathsf{in}\ t \ \longrightarrow_\zeta\ t[v/x]$

CIC enjoys the property that definitional equality is decidable, which is essential for typechecking to be decidable.

2.4 Inductive Types

An inductive type declaration specifies a family of mutually defined types and their constructors. Formally, an inductive specification is a tuple $\mathsf{Ind}\{p, \Gamma_I, \Gamma_C\}$ where p is the number of parameters, Γ_I declares the type constructors, and Γ_C declares the data constructors [11]. The *strict positivity* constraint ensures that recursive occurrences of the type being defined appear only in strictly positive positions in constructor argument types, guaranteeing logical consistency.

$$
\begin{array}{c}
\vdots \\
\uparrow : \\
\mathsf{Type}(2) \\
\uparrow : \\
\mathsf{Type}(1) \\
\uparrow : \\
\mathsf{Type}(0) \\
\uparrow : \\
\mathsf{Prop}
\end{array}
$$

Figure 2: The universe hierarchy. Each sort inhabits the next, forming a cumulative tower.

Example 2.1 (Natural numbers)**.** The natural numbers are defined inductively:

$$\mathsf{Ind}\{0,\ (\mathsf{Nat} : \mathsf{Type}),\ (\mathsf{zero} : \mathsf{Nat},\ \mathsf{succ} : \mathsf{Nat} \to \mathsf{Nat})\}$$

The elimination principle for Nat yields primitive recursion.

Example 2.2 (Propositional equality)**.** The identity type Eq is an inductive family:

$$\mathsf{Ind}\{1,\ (\mathsf{Eq} : \Pi(A{:}\mathsf{Type}).\, A \to A \to \mathsf{Prop}),\ (\mathsf{refl} : \Pi(A{:}\mathsf{Type}).\, \Pi(a{:}A).\, \mathsf{Eq}\ A\ a\ a)\}$$

2.5 Universe Polymorphism

Modern implementations of CIC (including Lean 4) support *universe polymorphism*: definitions can be parameterized by universe level variables, instantiated at each use site. This avoids code duplication and is essential for a practical foundation.

The universe hierarchy is:

$$\mathsf{Prop}\ :\ \mathsf{Type}(0)\ :\ \mathsf{Type}(1)\ :\ \mathsf{Type}(2)\ :\ \cdots$$

Universe constraints (e.g., $u \leq v$, $u+1 \leq v$) are collected during typechecking and must be satisfiable. Universe polymorphism interacts importantly with content-addressing (Section 7.2).

3 Content-Addressed Terms

3.1 The Naming Problem in Detail

Consider the theorem that addition on natural numbers is commutative. In Lean 4's Mathlib, this is `Nat.add_comm`. In Rocq's standard library, it is also `Nat.add_comm`—but the two are not interchangeable because they exist in different systems with different elaboration, different dependencies, and different internal representations. An AI agent might prove the same theorem and call it `theorem_7a3f`. Three correct formalizations of the same mathematical fact, none of which can reference the others.

The problem compounds at scale. Mathlib has undergone multiple renaming campaigns, where thousands of theorems are renamed to conform to evolving conventions. Each renaming breaks downstream projects that depend on the old names. When independent groups formalize overlapping mathematics, reconciliation requires human labor to identify duplicates, choose canonical names, and redirect references.

3.2 Content-Addressing for CIC Terms

We define the hash function $\mathcal{H}$ on CIC terms recursively. All hashing uses SHA3-256 [16]. Tag strings ensure distinct term constructors never collide.

Definition 3.1 (Term hash)**.** The hash function $\mathcal{H} : \mathsf{Term} \to \{0,1\}^{256}$ is defined by:

$$\begin{aligned}
\mathcal{H}(\mathsf{bvar}(i)) &= \mathsf{SHA3}(\texttt{"bvar"} \parallel \mathsf{encode}(i)) \\
\mathcal{H}(\mathsf{Sort}(u)) &= \mathsf{SHA3}(\texttt{"sort"} \parallel \mathcal{H}_{\mathsf{level}}(u)) \\
\mathcal{H}(\mathsf{const}(n, \bar{u})) &= \mathsf{SHA3}(\texttt{"const"} \parallel \mathcal{H}(\mathsf{defn}(n)) \parallel \mathcal{H}_{\mathsf{levels}}(\bar{u})) \\
\mathcal{H}(\mathsf{app}(f, a)) &= \mathsf{SHA3}(\texttt{"app"} \parallel \mathcal{H}(f) \parallel \mathcal{H}(a)) \\
\mathcal{H}(\mathsf{lam}(_, T, b, _)) &= \mathsf{SHA3}(\texttt{"lam"} \parallel \mathcal{H}(T) \parallel \mathcal{H}(b)) \\
\mathcal{H}(\mathsf{forallE}(_, T, b, _)) &= \mathsf{SHA3}(\texttt{"forallE"} \parallel \mathcal{H}(T) \parallel \mathcal{H}(b)) \\
\mathcal{H}(\mathsf{letE}(_, T, v, b, _)) &= \mathsf{SHA3}(\texttt{"letE"} \parallel \mathcal{H}(T) \parallel \mathcal{H}(v) \parallel \mathcal{H}(b))
\end{aligned}$$

where $\mathsf{defn}(n)$ resolves the constant n to its full definition (type and value), and _ denotes fields excluded from the hash (binder names, binder info).

Several design decisions are critical:

Binder names are excluded. The terms $\lambda(x{:}\mathsf{Nat}).\, x$ and $\lambda(y{:}\mathsf{Nat}).\, y$ are α-equivalent and receive the same hash. Since we use de Bruijn indices internally, the binder name is already redundant—it is carried only for pretty-printing. Excluding it means α-equivalence is built into the hash by construction.

Binder info is excluded. Lean 4 attaches *binder info* to each binder: whether the argument is explicit, implicit, strict implicit, or instance implicit. This is an elaboration concern, not a type-theoretic one. We exclude binder info from the core hash; elaboration metadata can be stored separately as annotations on the hash.

Constants resolve to definitions. The hash of $\mathsf{const}(n, \bar{u})$ includes the hash of the *definition* that n refers to, not the string n itself. This is the key move that eliminates name-dependence: two constants with different names but identical definitions produce the same hash.

Free variables are forbidden. Only *closed* terms—those with no free variables—may be stored in the hashset. Every bound variable must be under a binder, and every constant must resolve to a stored definition.

Example 3.1 (Hashing the identity function on Nat)**.** Consider $\lambda(n{:}\mathsf{Nat}).\, n$, which in de Bruijn notation is $\mathsf{lam}(_, \mathsf{Nat}, \mathsf{bvar}(0), _)$. Its hash is:

$$\mathcal{H}(\mathsf{lam}(_, \mathsf{Nat}, \mathsf{bvar}(0), _)) = \mathsf{SHA3}(\texttt{"lam"} \parallel \mathcal{H}(\mathsf{Nat}) \parallel \mathsf{SHA3}(\texttt{"bvar"} \parallel \mathsf{encode}(0)))$$

where $\mathcal{H}(\mathsf{Nat})$ is the hash of the inductive definition of natural numbers.

Example 3.2 (Alpha-equivalence)**.** The terms $\lambda(x{:}\mathsf{Nat}).\, x$ and $\lambda(y{:}\mathsf{Nat}).\, y$ both have de Bruijn representation $\mathsf{lam}(_, \mathsf{Nat}, \mathsf{bvar}(0), _)$. Since binder names are excluded from the hash, they produce identical hashes. □

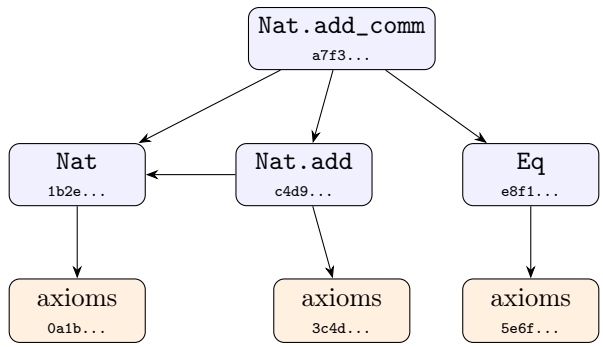


Figure 3: Merkle DAG of `Nat.add_comm`. Each node's hash transitively encodes all dependencies down to the axioms. Human-readable names shown for clarity; the system operates on hashes.

3.3 Capturing the Environment

Definition 3.2 (Hash-closed term)**.** A term t is *hash-closed* if every constant reference $\mathsf{const}(n, \bar{u})$ occurring in t can be resolved to a declaration whose hash exists in the hashset $\mathcal{S}$.

Hash-closure means that the hash of any term transitively encodes its entire dependency tree. This creates a **Merkle DAG** of mathematical knowledge: the hash of `Nat.add_comm` encodes, through the hashes of its subterms, the definitions of Nat, Nat.add, Eq, and transitively down to the axioms of the type theory.

3.4 Mutual Recursive Definitions

Mutual recursion presents a challenge: if definition A references B and B references A, then $\mathcal{H}(A)$ depends on $\mathcal{H}(B)$ and vice versa.

Definition 3.3 (Mutual block hash)**.** A mutual recursive block $M = (d_1, \ldots, d_k)$ is an ordered list of definitions that may reference each other. The block hash is:

$$\mathcal{H}(M) = \mathsf{SHA3}(\texttt{"mutual"} \parallel \mathcal{H}_{\mathsf{raw}}(d_1) \parallel \cdots \parallel \mathcal{H}_{\mathsf{raw}}(d_k))$$

where $\mathcal{H}_{\mathsf{raw}}(d_i)$ hashes definition d_i with intra-block references represented as block-relative indices rather than hashes. Each individual definition's identity is then the pair $(\mathcal{H}(M), i)$ where i is its index within the block.

This is analogous to how Lean 4 handles `mutual ... end` blocks. The approach preserves the acyclic Merkle DAG property at the block level: blocks reference other blocks (or individual definitions), and no cyclic block dependencies are permitted.

3.5 The Global Hashset

Definition 3.4 (Hashset)**.** The global hashset is a set of verified triples:

$$\mathcal{S} = \{(h, t, T) \mid h = \mathcal{H}(t),\ T = \mathsf{typeof}(t),\ \mathsf{well_typed}(t)\}$$

together with a *type index*:

$$\mathcal{I} : \{0,1\}^{256} \to \mathcal{P}(\{0,1\}^{256}), \qquad \mathcal{I}(h_T) = \{h \mid (h, t, T) \in \mathcal{S},\ \mathcal{H}(T) = h_T\}$$

mapping each type hash to the set of hashes of its inhabitants.

The hashset is *append-only*: once a verified triple is added, it is never removed. Since well-typedness is verified before insertion, every entry in the hashset is correct by construction.

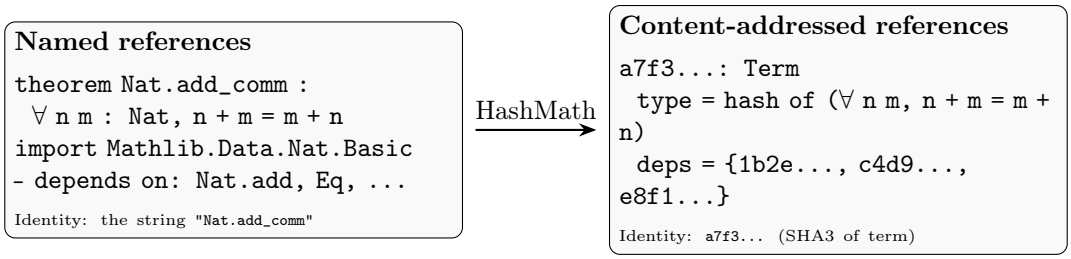


Figure 4: Named references (left) identify terms by human-chosen strings within a specific library. Content-addressed references (right) identify terms by their cryptographic hash, independent of any naming convention.

3.6 Comparison with Unison

The Unison programming language pioneered content-addressing for general-purpose code, hashing abstract syntax trees with 512-bit SHA3 [14]. HashMath adapts this approach for the specific demands of formal mathematics:

- **Type-to-inhabitants index**: Unison does not maintain a global map from types to their inhabitants, because general-purpose code does not have the same "search by type" need.
- **Formal verification**: every term in HashMath is type-checked before insertion; Unison stores arbitrary well-scoped code without a comparable verification step.
- **Universe polymorphism**: CIC's universe polymorphism requires careful treatment in the hash function (Section 7.2).
- **Inductive types**: CIC's inductive types, with strict positivity and elimination principles, have no analog in Unison's type system.

Unison's analysis of SHA3 collision probability applies equally to HashMath: with 2^{128} security against collision attacks, the probability of a collision in a library of 2^{40} terms (one trillion) is negligible ($\approx 2^{-176}$).

4 Verification and Trust

4.1 The Verification Protocol

When a term t is submitted to the hashset, the following protocol executes:

1. **Parse**: deserialize t from the binary format into a CIC term.
2. **Resolve**: verify that every constant reference in t has a corresponding entry in $\mathcal{S}$.
3. **Typecheck**: type-check t in the CIC, using the referenced definitions as the environment. If t is ill-typed, reject.
4. **Hash**: compute $h = \mathcal{H}(t)$ and $h_T = \mathcal{H}(\mathsf{typeof}(t))$.
5. **Store**: add $(h, t, \mathsf{typeof}(t))$ to $\mathcal{S}$ and h to $\mathcal{I}(h_T)$.

If any step fails, the term is rejected and no state changes. The protocol is *deterministic*: given the same hashset state and the same term, every compliant node produces the same result.

4.2 Trust Properties

The verification protocol establishes several properties:

Theorem 4.1 (Correctness)**.** *Every term $(h, t, T) \in \mathcal{S}$ satisfies: t is well-typed in CIC with type T, and all dependencies of t are also in $\mathcal{S}$.*

This follows directly from the protocol: step 3 verifies well-typedness, step 2 verifies that all dependencies exist, and the append-only nature of the hashset ensures that dependencies are never invalidated.

Additional trust properties:

- **Integrity**: SHA3-256 collision resistance ensures that distinct terms receive distinct hashes, with overwhelming probability.
- **Immutability**: a hash pins exact content; the term at hash h can never change.
- **Composability**: any well-typed term can reference any existing term in $\mathcal{S}$ by hash, regardless of who contributed either term.
- **No trust in contributors**: verification is mechanical. A term submitted by an untrusted AI agent is verified identically to one submitted by a Fields medalist—and both are equally trustworthy once verified.

4.3 The Small Trusted Computing Base

The system's correctness depends only on:

1. the CIC type checker being correct,
2. the SHA3 implementation being correct, and
3. the serialization format faithfully representing terms.

The CIC type checker is remarkably small. Diehl [12] demonstrates that dependent type checking can be implemented in approximately 200 lines of Python, and his "Typechecker Zoo" [13] catalogs minimal implementations across languages. This is the *de Bruijn criterion* [17]: a small, auditable kernel through which all trust flows.

Crucially, this kernel can itself be formally verified. Sozeau et al. [19] demonstrated in the MetaCoq project that the CIC type checker can be formalized *within CIC itself* and proved correct—"Coq Coq Correct!" Their verified type checker, proved sound with respect to a formal specification of CIC, found and fixed a real incompleteness bug in Coq 8.14. This shifts the trust from a *trusted code base* (TCB) to a *trusted theory base* (TTB): one need not trust the implementation, only the mathematical specification of the type theory.

HashMath could adopt MetaCoq's verified checker, or a similar verified kernel, to minimize the trusted computing base to the bare mathematical theory.

Remark 4.1. The de Bruijn criterion was articulated by Pollack [18]: "How to believe a machine-checked proof." The answer is to make the proof-checking kernel so small that it can be inspected and verified independently of the proof-generating machinery.

5 The Knowledge Base Architecture

5.1 Data Model

The knowledge base consists of three components:

$$\begin{aligned}
\mathcal{S} &= \{(h, t, T) \mid h = \mathcal{H}(t),\ T = \mathsf{typeof}(t),\ \mathsf{well_typed}(t)\} \\
\mathcal{I} &= \{h_T \mapsto \{h \mid (h, t, T) \in \mathcal{S},\ \mathcal{H}(T) = h_T\}\} \\
\mathcal{N} &= \{\mathit{name} \mapsto h\} \quad \text{(metadata, not core)}
\end{aligned}$$

The hashset and type index are the core infrastructure; name maps are an optional layer maintained by communities or individuals.

5.2 Operations

The knowledge base supports five primary operations:

- $\mathsf{submit}(t)$: verify t and add to $\mathcal{S}$; return $\mathcal{H}(t)$.
- $\mathsf{lookup}(h)$: retrieve the term and type at hash h.
- $\mathsf{search_by_type}(h_T)$: return $\mathcal{I}(h_T)$—all inhabitants of type T.
- $\mathsf{dependencies}(h)$: compute the transitive dependency set by walking the Merkle DAG.
- $\mathsf{name}(h, n)$: associate human-readable name n with hash h (metadata operation).

5.3 Storage and Distribution

The hashset is designed for decentralized storage and distribution:

- **Replication**: any node can maintain a full or partial copy of $\mathcal{S}$, similar to Git or IPFS [26].
- **Independent verification**: any node can verify any term independently, requiring only the term's transitive dependencies.
- **No central authority**: there is no privileged node; all nodes are equal.
- **Incremental syncing**: a node need only fetch hashes it does not already have. Since hashes are deterministic, two nodes with the same hash have identical content.

Terms are serialized in a compact binary format for storage and transmission. The serialization must be canonical (deterministic) to ensure that hash computation is consistent across implementations.

5.4 The Type-to-Inhabitants Index

The type index $\mathcal{I}$ enables a powerful new mode of mathematical search. Given a type—which is itself a proposition under the Curry-Howard correspondence—one can ask: "what are all known proofs of this proposition?"

Example 5.1. To find all proofs that $\forall(n : \mathsf{Nat}),\ n + 0 = n$, one computes the hash of the type $\Pi(n{:}\mathsf{Nat}).\,\mathsf{Eq\ Nat}\ (\mathsf{Nat.add}\ n\ \mathsf{Nat.zero})\ n$ and queries $\mathcal{I}$ for that hash. The result includes every proof of this fact in the entire knowledge base, regardless of who contributed it or what they named it.

This is analogous to Hoogle for Haskell or Moogle for Lean, but operating globally across all contributions. For AI agents, the type index transforms proof search from a name-recall problem into a type-matching problem—a far more natural interface for machine reasoning.

5.5 Interoperability Layer

HashMath is designed to interoperate with existing proof assistants through export pipelines:

Lean 4 pipeline. Lean 4's kernel expressions are CIC terms. An export tool translates Lean `Expr` values to HashMath's canonical representation, computes hashes, and submits to the hashset. This pipeline can import the entirety of Mathlib.

Rocq pipeline. Rocq's Gallina is also CIC. The differences between Lean's and Rocq's kernel terms are largely syntactic: both have the same term constructors, the same universe hierarchy, and the same inductive type scheme. A Rocq export tool performs the analogous translation.

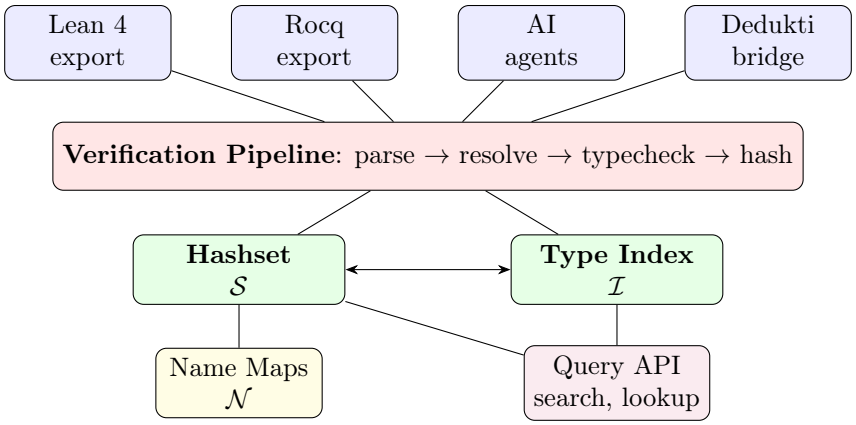


Figure 5: HashMath architecture. Terms from multiple sources pass through the verification pipeline before being added to the hashset and type index. Name maps are optional metadata. The query API supports lookup by hash and search by type.

Contrast with Logipedia/Dedukti. The Logipedia project [20] and the Dedukti logical framework [21] take a different approach: they translate proofs between different type theories through a common framework (the $\lambda\Pi$-calculus modulo theory). The EuroProofNet COST action [22] (2022–2025, 600+ researchers, 47 countries) has advanced this vision. HashMath bypasses the translation problem by working at the CIC kernel level where Lean and Rocq already agree. The approaches are complementary: Dedukti could translate non-CIC proofs (e.g., from Isabelle/HOL) into CIC for HashMath storage.

6 AI and Human Collaboration

6.1 The Coordination Problem Today

Tao's formalization of the Polynomial Freiman-Ruzsa conjecture in Lean involved over 20 contributors, with manual coordination through GitHub issues, Zulip chat, and personal communication [5]. Mathlib contributions require human review, conformance to naming conventions, and compliance with style linters. These processes ensure quality but limit throughput.

AI systems like AlphaProof [3] and DeepSeek-Prover-V2 [4] can produce correct proofs at machine speed, but cannot contribute them to shared libraries without human intermediation: someone must name the theorem, choose the right namespace, write documentation, and submit a pull request. The Polymath project [6] demonstrated the vision of permissionless mathematical collaboration—40+ contributors proving the density Hales-Jewett theorem through blog comments—but the coordination was entirely ad hoc.

6.2 HashMath Eliminates Coordination

In HashMath, the contribution workflow is:

1. Agent (AI or human) constructs a well-typed CIC term t.
2. Agent submits t to the hashset.
3. The verification pipeline checks t and computes $\mathcal{H}(t)$.
4. t is available globally, immediately, by hash.

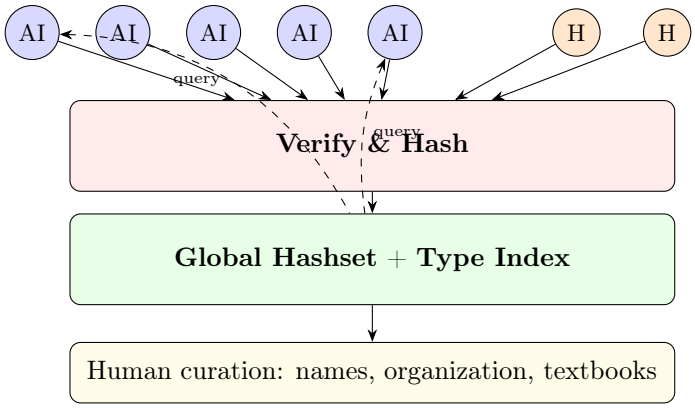


Figure 6: AI agents and humans contribute proof terms to the verification pipeline. Verified terms enter the global hashset. Agents discover existing results through type-directed queries. Humans add curation—names, organization, exposition—as an optional layer.

No pull request. No naming debate. No review queue. Another agent that needs a lemma of type T queries $\mathcal{I}(\mathcal{H}(T))$ and finds t by its type, not by any name.

When multiple agents independently prove the same theorem with the same proof term, they produce the same hash—the duplicate is automatically detected. When they produce different proofs of the same theorem, both are stored and discoverable through the type index. Neither outcome wastes work.

6.3 The Naming Layer as Social Convention

Names remain useful—for human communication, documentation, textbooks, and teaching. In HashMath, names are metadata: a mapping from strings to hashes, maintained by communities.

- The French school: "théorème de Bézout" $\mapsto$ `a7f3...`
- The English school: "Bezout's theorem" $\mapsto$ `a7f3...`
- An AI agent: `h_7f3a2b` $\mapsto$ `a7f3...`

All three reference the same hash. Name collisions are impossible because names are not identity—they are aliases. Different communities can maintain their own naming conventions without conflict.

6.4 Scaling Mathematics

Tao's vision of "big theorems proved by 20 people and multiple AIs" requires infrastructure that does not exist today. HashMath provides it.

Imagine 1000 AI agents exploring different corners of mathematics simultaneously. Each agent submits verified terms to the hashset. Other agents discover these terms by type signature and build on them. Human mathematicians curate, name, and organize the accumulating knowledge—but the raw content grows permissionlessly, limited only by the rate at which correct proofs can be produced.

7 Challenges and Limitations

We address the principal challenges honestly.

7.1 Definitional Equality and Hash Diversity

Two CIC terms that are *definitionally equal* but *syntactically different* receive different hashes. For example, $(\lambda(x{:}\mathsf{Nat}).\, x)\ \mathsf{zero}$ and zero are definitionally equal but hash differently. This is by design: the hash captures the exact proof term, not just the proposition it proves. The type index is the correct mechanism for finding "equivalent" results—all proofs of a given type are discoverable regardless of their syntactic form.

One could normalize terms before hashing to reduce diversity, but normalization is computationally expensive (exponential in the worst case) and would change the semantics: distinct proof strategies would become indistinguishable. We judge that hashing terms as-is, combined with the type index for discovery, is the right tradeoff.

7.2 Universe Polymorphism

Universe-polymorphic definitions are parameterized by universe level variables. The hash must capture universe parameters correctly: a definition instantiated at $\mathsf{Type}(0)$ and the same definition instantiated at $\mathsf{Type}(1)$ are different terms with different hashes. The hash function handles this through $\mathcal{H}_{\mathsf{level}}$ and $\mathcal{H}_{\mathsf{levels}}$, which hash universe level expressions (variables, successors, max) structurally. Care is needed to ensure that universe constraints are consistently captured.

7.3 Axiom Compatibility

Different mathematical traditions employ different axioms: classical logic vs. constructive logic, the axiom of choice, propositional extensionality, function extensionality, quotient types, and so on. In HashMath, axiom dependencies are carried transitively through the Merkle DAG. Every axiom used in a proof is visible by inspecting the transitive dependencies of its hash.

This enables powerful filtering: "show me all proofs of X that do not use the axiom of choice" is a computable query—walk the Merkle DAG and check whether the axiom of choice's hash appears in the dependency set. Different communities can maintain different axiom policies without preventing contribution to the shared hashset.

7.4 Hash Stability Across Format Versions

If the hashing scheme is ever modified—for example, to add support for a new term constructor—all hashes change. The Unison project experienced this challenge in migrating between hash format versions [14].

We mitigate this through several mechanisms:

- **Version prefix**: every hash includes a format version byte, so terms self-identify their hashing era.
- **Bidirectional mappings**: during transitions, maintain a mapping between old and new hashes.
- **Stability as a goal**: the hash format should be designed carefully and changed rarely. Getting it right early is a priority.

7.5 Mutual Recursion

Block-level hashing (Section 3.4) ensures acyclicity but introduces granularity concerns. If a mutual block is large, any modification to one definition changes the block hash and thus the identity of all definitions in the block. In practice, mutual blocks in CIC tend to be small (2–5 definitions),

making this manageable. Refactoring a mutual block into a non-mutual formulation (when possible) is a content-level change and correctly results in a different hash.

7.6 Performance

Hashing deep dependency trees requires traversing large Merkle DAGs. Several standard techniques address this:

- **Caching**: once a term's hash is computed, it is stored and never recomputed.
- **Memoization**: shared subterms (which are common in CIC terms) are hashed once.
- **Incremental verification**: when verifying a new term, only the new term and its immediate definition need type-checking; dependencies have already been verified.

7.7 Social Adoption

A content-addressed knowledge base is only useful if it contains content. This is a chicken-and-egg problem with a clear bootstrap strategy:

- **Import Mathlib**: Lean 4's Mathlib contains 210,000+ theorems and is CIC-based. Importing it provides a substantial initial corpus.
- **AI agent tooling**: provide APIs and tooling that make it easy for AI proving systems to submit and query the hashset.
- **Compatibility**: HashMath is complementary to existing systems, not a replacement. Lean and Rocq users can continue using their preferred tools; HashMath is the storage and discovery layer.

8 Related Work

8.1 Proof Assistants

Lean 4 [1], Rocq (formerly Coq) [8], Agda, and Isabelle/HOL are the dominant proof assistants. All use named references within curated libraries. HashMath is complementary: a content-addressed storage layer that any CIC-based system can read from and write to, without modifying the system itself.

8.2 Unison

Unison [14] is the closest precedent. It content-addresses general-purpose functional programs using SHA3 hashes of abstract syntax trees. HashMath adapts this idea for the Calculus of Inductive Constructions, adding the type-to-inhabitants index, requiring formal verification before storage, and handling CIC-specific features (universe polymorphism, inductive types, strict positivity).

8.3 Logipedia and Dedukti

The Logipedia project [20] aims to build a universal library of formal proofs by translating between systems through the Dedukti logical framework [21], based on the $\lambda\Pi$-calculus modulo theory. The EuroProofNet COST action [22] (2022–2025) has advanced this vision with over 600 researchers across 47 countries. Dedukti translates between *different type theories* through a common framework; HashMath works at the CIC kernel level where Lean and Rocq *already agree*. The approaches are complementary: Dedukti could translate non-CIC proofs into CIC for HashMath storage.

8.4 MetaCoq

MetaCoq [19] formalizes the Calculus of Inductive Constructions within Coq itself, including a verified type checker proved sound with respect to a formal specification. This demonstrates that the CIC kernel—the core of HashMath's trusted computing base—can be formally verified. HashMath could adopt MetaCoq's verified checker to minimize trust assumptions.

8.5 IPFS and Content-Addressed Storage

IPFS [26] provides content-addressed storage for arbitrary data. HashMath's storage model is similar but domain-specific: it verifies terms before storing them (IPFS stores any data) and maintains a type index that is meaningful only for CIC terms. HashMath could use IPFS as an underlying storage layer.

8.6 Formal Mathematics Libraries

Mathlib [2], the Archive of Formal Proofs, and Rocq's standard library are curated, centralized, and name-dependent. They represent enormous investments of human effort. HashMath is uncurated, decentralized, and name-independent—but can import these libraries wholesale as its initial corpus.

8.7 AI for Mathematics

AlphaProof [3], DeepSeek-Prover-V2 [4], LeanDojo, and COPRA produce proofs within existing proof assistants. HashMath provides a system-agnostic destination for AI-generated proofs, and the type index provides a natural interface for AI proof search: search by type signature rather than by name.

9 Roadmap

We outline a phased development plan.

Phase 1: Core Infrastructure. Implement the CIC type checker—following Diehl [12], the core can be realized in hundreds of lines—the hash function, the hashset with type index, and a canonical binary serialization format.

Phase 2: Import Pipeline. Build a Lean 4 export pipeline that translates `Expr` to HashMath terms, hashes them, and populates the hashset. Import Mathlib as the initial corpus (210,000+ theorems). Build a Rocq export pipeline for the standard library.

Phase 3: AI Integration. Provide an API for AI agents to submit verified terms and query the type index. Build a type-directed search interface. Integrate with existing AI proving systems (DeepSeek-Prover, LeanDojo, etc.).

Phase 4: Distribution and Scaling. Implement peer-to-peer replication of the hashset. Optimize incremental syncing (transfer only missing hashes). Optimize query performance for the type index at scale.

Phase 5: Ecosystem. Build IDE integration (VS Code, Emacs) for browsing and searching the hashset. Build a web interface for exploration. Support community-maintained name maps. Develop proof visualization tools.

10 Conclusion

The Calculus of Inductive Constructions is the right foundation for a global mathematical knowledge base: it is the shared core of the most successful proof assistants, it is expressive enough for serious mathematics, and its type checker is small enough to be formally verified.

Content-addressing eliminates the coordination bottleneck that limits the scale of formalized mathematics. No naming consensus, no centralized curation, no review queues—just verified terms, identified by their structure, composable by hash.

The type-to-inhabitants index turns the knowledge base into a searchable library where AI agents and human mathematicians discover relevant results by type signature, not by remembering names.

HashMath provides the infrastructure for a future in which thousands of AI agents and hundreds of mathematicians contribute simultaneously, building on each other's work without a single naming conflict or a single gatekeeper.

Mathematics should be permissionless. HashMath makes it so.

References

[1] L. de Moura and S. Ullrich, "The Lean 4 theorem prover and programming language," in *Proc. CADE-28*, 2021, pp. 625–635.

[2] The Mathlib Community, "Mathlib4," https://github.com/leanprover-community/mathlib4, 2024.

[3] AlphaProof and AlphaGeometry Teams, "AI achieves silver-medal standard solving International Mathematical Olympiad problems," *Nature*, 2025.

[4] Z. Ren et al., "DeepSeek-Prover-V2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition," *arXiv:2504.21801*, 2025.

[5] T. Tao, "Machine-assisted proofs," *ICM Invited Lecture*, 2025.

[6] W. T. Gowers and M. Nielsen, "Massively collaborative mathematics," *Nature*, vol. 461, pp. 879–881, 2009.

[7] D. H. J. Polymath, "A new proof of the density Hales-Jewett theorem," *Annals of Mathematics*, vol. 175, no. 3, pp. 1283–1327, 2012.

[8] T. Coquand and G. Huet, "The Calculus of Constructions," *Information and Computation*, vol. 76, pp. 95–120, 1988.

[9] T. Coquand and C. Paulin-Mohring, "Inductively defined types," in *Proc. COLOG-88*, LNCS 417, Springer, 1990.

[10] B. Werner, "Une théorie des constructions inductives," Ph.D. thesis, Université Paris 7, 1994.

[11] C. Paulin-Mohring, "Inductive definitions in the system Coq: Rules and properties," in *Proc. TLCA*, LNCS 664, Springer, 1993.

[12] L. Diehl, "Dependent types in 200 lines of Python," blog post, 2023.

[13] L. Diehl, "The Typechecker Zoo," https://github.com/larrytheliquid/typecheckerzoo, 2023.

[14] P. Chiusano and R. Bjarnason, "Unison: A friendly programming language from the future," https://www.unison-lang.org/, 2024.

[15] R. C. Merkle, "A certified digital signature," in *Proc. CRYPTO'89*, LNCS 435, Springer, 1990. (Original concept, 1979.)

[16] NIST, "SHA-3 Standard: Permutation-based hash and extendable-output functions," *FIPS PUB 202*, 2015.

[17] N. G. de Bruijn, "The mathematical language AUTOMATH, its usage, and some of its extensions," in *Symposium on Automatic Demonstration*, LNCS 125, Springer, 1970.

[18] R. Pollack, "How to believe a machine-checked proof," in *Twenty-Five Years of Constructive Type Theory*, Oxford, 1998.

[19] M. Sozeau, S. Boulier, Y. Forster, N. Tabareau, and T. Winterhalter, "Coq Coq correct! Verification of type checking and erasure for Coq, in Coq," *Proc. ACM Program. Lang.*, vol. 4 (POPL), pp. 8:1–8:28, 2020.

[20] G. Dowek, "Logipedia: a multi-system encyclopedia of formal proofs," *arXiv:2305.00064*, 2023.

[21] A. Assaf, G. Burel, R. Cauderlier, D. Delahaye, G. Dowek, C. Dubois, F. Gilbert, P. Halmagrand, O. Hermant, and R. Saillard, "Dedukti: a logical framework based on the $\lambda\Pi$-calculus modulo theory," 2016.

[22] F. Blanqui et al., "EuroProofNet: European research network on digital proofs," COST Action CA20111, 2022–2025.

[23] G. Gonthier, "Formal proof—the Four Color Theorem," *Notices of the AMS*, vol. 55, no. 11, pp. 1382–1393, 2008.

[24] G. Gonthier et al., "A machine-checked proof of the Odd Order Theorem," in *Proc. ITP*, LNCS 7998, Springer, 2013.

[25] X. Leroy, "Formal verification of a realistic compiler," *Commun. ACM*, vol. 52, no. 7, pp. 107–115, 2009.

[26] J. Benet, "IPFS—Content addressed, versioned, P2P file system," *arXiv:1407.3561*, 2014.

[27] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008.

[28] V. Buterin, "Ethereum: A next-generation smart contract and decentralized application platform," 2014.